EXPERIMENT - 76

Question 1

1. Problem Statement

Create an enumeration for user roles (ADMIN, USER, GUEST) and display messages based on role.
Input: GUEST
Output: Welcome Guest

2. Algorithm

Step 1: START
Step 2: Declare enum Role with ADMIN, USER and GUEST.
Step 3: Read the role and change its letters to capital letters.
Step 4: If it is ADMIN, USER or GUEST, set role to that value. Otherwise display Invalid role and stop.
Step 5: If role is ADMIN display Welcome Admin, if USER display Welcome User, and if GUEST display Welcome Guest.
Step 6: STOP

3. Test Case Table

#	Input	Expected Output	Actual Output	Result
1	GUEST	Welcome Guest	Welcome Guest	PASS
2	ADMIN	Welcome Admin	Welcome Admin	PASS
3	USER	Welcome User	Welcome User	PASS
4	guest	Welcome Guest	Welcome Guest	PASS
5	MANAGER	Invalid role	Invalid role	PASS

4. Source Code

role76.c

```
/* Experiment 76, Question 1: enumeration for user roles. */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

enum Role { ADMIN, USER, GUEST };

int main(void)
{
    char input[20];
    enum Role role;
    int i;

    printf("Enter the role (ADMIN, USER or GUEST): ");
    scanf("%19s", input);
    for (i = 0; input[i] != '\0'; i++)
        input[i] = toupper((unsigned char)input[i]);

    if (strcmp(input, "ADMIN") == 0)
        role = ADMIN;
    else if (strcmp(input, "USER") == 0)
        role = USER;
    else if (strcmp(input, "GUEST") == 0)
        role = GUEST;
    else {
        printf("Invalid role\n");
        return 1;
    }

    switch (role) {
    case ADMIN:
        printf("Welcome Admin\n");
        break;
    case USER:
        printf("Welcome User\n");
        break;
    case GUEST:
        printf("Welcome Guest\n");
        break;
    }
    return 0;
}
```

Question 2

1. Problem Statement

Print all enum names and integer values of 12 colors using a loop.
Input: No Input
Output: RED=0, BLUE=1, GREEN=2, and so on for all 12 colors

2. Algorithm

Step 1: START
Step 2: Declare enum Color with 12 colors: RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE, PINK, BROWN, BLACK, WHITE, GRAY and CYAN, so their values are 0 to 11.
Step 3: Repeat for color = RED to CYAN: display the name and the integer value, with a comma after every color except CYAN.
Step 4: STOP

3. Test Case Table

#	Input	Expected Output	Actual Output	Result
1	Nil	RED=0, BLUE=1, GREEN=2, YELLOW=3, ORANGE=4, PURPLE=5, PINK=6, BROWN=7, BLACK=8, WHITE=9, GRAY=10, CYAN=11	RED=0, BLUE=1, GREEN=2, YELLOW=3, ORANGE=4, PURPLE=5, PINK=6, BROWN=7, BLACK=8, WHITE=9, GRAY=10, CYAN=11	**PASS**

4. Source Code

color76.c

```
/* Experiment 76, Question 2: names and values of 12 colors using a loop. */
#include <stdio.h>

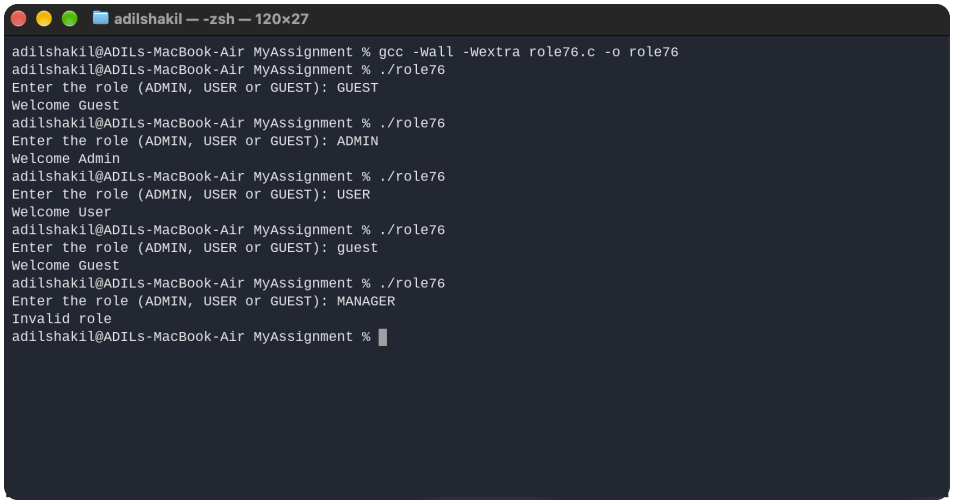
enum Color { RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE,
             PINK, BROWN, BLACK, WHITE, GRAY, CYAN };

int main(void)
{
    const char *names[] = {"RED", "BLUE", "GREEN", "YELLOW", "ORANGE", "PURPLE",
                           "PINK", "BROWN", "BLACK", "WHITE", "GRAY", "CYAN"};
    enum Color color;

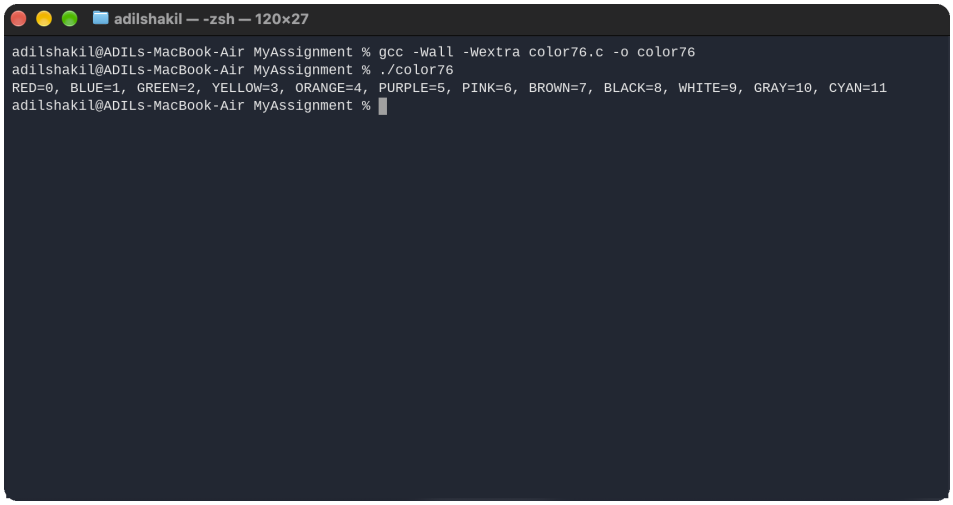
    for (color = RED; color <= CYAN; color++) {
        printf("%s=%d", names[color], color);
        if (color < CYAN)
            printf(", ");
    }
    printf("\n");
    return 0;
}
```

5. Compilation and Output

Question 1: compiled with `gcc -Wall -Wextra role76.c -o role76`



Question 2: compiled with `gcc -Wall -Wextra color76.c -o color76`



Both programs built clean, and the first test of Question 1 is the example from the question.